PRACTICAL NO:- 05

CM25D008

Output :-

Consider there are N philosophers seated around a circular table with one chopstick between each pair of philosophers.

- Each philosopher needs two chopsticks to eat.
- A philosopher may eat only if he can pick up both adjacent chopsticks.
- One chopstick can be picked up by only one of its adjacent philosophers at a time (not both).

Write a program to solve the problem using **process synchronization technique**.

Code:-

```
#include <stdio.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>

#define N 5

sem_t chopstick[N];
void* philosopher(void* num)
{

int id = *(int*)num;

while(1)
{
printf("Philosopher %d is Thinking\n",id);
sleep(1);
```

```
void* philosopher(void* num)
{

int id = *(int*)num;

while(1)
{
printf("Philosopher %d is Thinking\n",id);
sleep(1);

sem_wait(&chopstick[id]);
printf("Philosopher %d picked Right Chopstick\n",id);

printf("Philosopher %d is Eating\n",id);
sleep(2);

sem_post(&chopstick[id]);
```

```
}

int main()
{
pthread_t phil[N];
int id[N];

for(int i =0; i<N;i++)
sem_init(&chopstick[i],0,1);

for(int i=0;i<N;i++){
id[i] = i;
pthread_create(&phil[i], NULL,philosopher,&id[i]);
}

for(int i=0; i<N;i++)
pthread_join(phil[i],NULL);
```

Output:-

```
ubuntu@sejal:~$ gcc dining.c -o dining -lpthread
ubuntu@sejal:~$ ./dining
Philosopher 0 is Thinking
Philosopher 1 is Thinking
Philosopher 2 is Thinking
Philosopher 3 is Thinking
Philosopher 4 is Thinking
Philosopher 1 picked Right Chopstick
Philosopher 1 is Eating
Philosopher 0 picked Right Chopstick
Philosopher 0 is Eating
Philosopher 4 picked Right Chopstick
Philosopher 4 is Eating
Philosopher 3 picked Right Chopstick
Philosopher 3 is Eating
Philosopher 2 picked Right Chopstick
Philosopher 2 is Eating
Philosopher 1 finished Eating
Philosopher 1 is Thinking
Philosopher 0 finished Eating
Philosopher 0 is Thinking
```